Java Collections: Internals and Contracts - Cheat Sheet

HashMap internals

HashMap is the centerpiece of every Java interview, and for good reason. The internals show up in almost every other collection (LinkedHashMap, HashSet, ConcurrentHashMap all build on it).

The bucket array

At its core, HashMap is a single array called `table`:

```
transient Node<K,V>[] table;

static class Node<K,V> implements Map.Entry<K,V> {
    final int hash;
    final K key;
    V value;
    Node<K,V> next;  // for collision chains
}
```

Each slot in `table` is called a bucket. The array starts empty and lazily allocates on the first `put()`.

Default sizes and why they matter

```
static final int DEFAULT_INITIAL_CAPACITY = 1 << 4;   // 16
static final float DEFAULT_LOAD_FACTOR = 0.75f;
static final int TREEIFY_THRESHOLD = 8;
static final int UNTREEIFY_THRESHOLD = 6;
static final int MIN_TREEIFY_CAPACITY = 64;
```

Initial capacity is 16 (the table size, not the entry count). Load factor is 0.75. Once `size > capacity * 0.75`, the table doubles. 16 with a 0.75 load factor means the first resize kicks in at 12 entries.

Why capacity must be a power of 2

HashMap doesn't use `hash % capacity`. It uses `hash & (capacity - 1)`. That bitwise AND runs much faster than modulo, but it only works if capacity is a power of 2.

```
// capacity = 16, so (n - 1) = 15 = 0b1111
int index = (n - 1) & hash;
```

If you pass `new HashMap<>(17)`, HashMap silently rounds up to 32. It enforces the power-of-2 invariant for you.

The hash() function

Java 8 added an XOR-shift step to the key's hash code:

```
static final int hash(Object key) {
    int h;
    return (key == null) ? 0 : (h = key.hashCode()) ^ (h >>> 16);
}
```

What's going on here: when you mask with `(n - 1)` and `n` is small (say 16), only the bottom 4 bits of the hash matter. The top 28 bits get thrown away. If two different keys happen to have the same low bits, they collide.

The XOR with the right-shifted version mixes the high bits into the low bits. So even keys whose hashCodes differ only in the upper bits end up in different buckets.

Bucket layout (ASCII)

```
table[0] -> null
table[1] -> Node(k1,v1) -> Node(k5,v5) -> null     (collision chain)
table[2] -> null
table[3] -> Node(k3,v3) -> null
table[4] -> [Red-Black Tree root]                  (treeified bucket)
            |
            ... 9+ entries arranged as a tree
table[5] -> null
...
```

Most buckets hold 0 or 1 entries. A bucket with 2+ entries forms a collision chain (linked list). A bucket with more than 8 entries (and a table of at least 64) converts that list into a red-black tree.

Collision handling: before and after Java 8

Pre-Java 8: every bucket was a linked list. Lookup in a degenerate case (everything hashes to the same bucket) was O(n).

Java 8+: once a bucket has more than `TREEIFY_THRESHOLD` (8) entries AND the table size is at least `MIN_TREEIFY_CAPACITY` (64), the linked list converts to a red-black tree. Lookup in that bucket becomes O(log n).

The 64-slot guard exists because if the table is tiny, the right fix is to resize, so treeifying a small table just hides the real problem.

When entries drop below `UNTREEIFY_THRESHOLD` (6), the tree converts back to a list. The hysteresis (8 vs 6) prevents thrashing.

Resizing and rehashing

When `size > capacity * loadFactor`, HashMap doubles the table:

```
// pseudo-code of resize()
Node<K,V>[] oldTab = table;
int oldCap = oldTab.length;
int newCap = oldCap << 1;     // double
Node<K,V>[] newTab = new Node[newCap];
// walk every entry, recompute index, place it in newTab
```

This is an O(n) operation. Java 8 has a clever trick: when capacity doubles, an entry's new index is either the same as before, or `oldIndex + oldCap`. You only need to look at one extra bit of the hash to decide. So the redistribution avoids recomputing the full mod.

If you know you're loading 10,000 entries, pre-size: `new HashMap<>(16384)` or use `Maps.newHashMapWithExpectedSize(10000)` (Guava). This avoids multiple resize cycles.

Why mutable keys break HashMap

```
HashMap<User, String> map = new HashMap<>();
User u = new User(1, "alice@example.com");
map.put(u, "active");

u.setEmail("alice2@example.com");  // mutates a field used in hashCode/equals

map.get(u);  // returns null. The bucket index now resolves to a different slot.
```

The entry is still physically in the map. You just can't find it. This is one of the most common HashMap bugs in real code. Treat keys as immutable, or at least don't mutate fields that participate in equals/hashCode after insertion.

Put/get walkthrough

```
Map<String, Integer> m = new HashMap<>();
// table is null

m.put("apple", 1);
// table allocated, size 16
// hash("apple") = ... let's say 0xC8E9F00D
// (h ^ (h >>> 16)) = 0xC8E9F00D ^ 0x0000C8E9 = 0xC8E938E4
// index = 0xC8E938E4 & 15 = 4
// table[4] = Node("apple", 1)

m.put("banana", 2);
// index = 7 (say). table[7] = Node("banana", 2)

m.put("cherry", 3);
// index = 4 (collision with "apple")
// table[4] = Node("apple", 1) -> Node("cherry", 3)

m.get("cherry");
// hash, mask -> bucket 4
// walk chain: "apple".equals("cherry") -> false
// next: "cherry".equals("cherry") -> true, return 3
```

The key thing to notice: `get()` does an `equals()` check at every link in the chain. A bad `equals()` method silently breaks `get()`.

ConcurrentHashMap internals (brief)

Full usage examples are in collections-concurrent.pdf. This is just the mechanism.

Java 7 design: lock striping

The table was split into 16 segments by default. Each segment was its own HashMap with its own lock. Writes locked only the relevant segment, so up to 16 threads could write at once (one per segment). Reads were mostly lock-free.

The downside: 16 segments was a fixed cap on write concurrency, and the memory overhead was high (each segment carried its own state).

Java 8+ redesign

The segments are gone. The structure is a single table, like regular HashMap. Concurrency works at the bucket level. Inserting into an empty bucket uses CAS (compare-and-swap) on the bucket head, no lock at all. Inserting into a non-empty bucket goes through `synchronized` on the bucket head node, which locks only that bucket. Reads need no lock, using volatile reads on the bucket head.

The result: concurrency scales with the number of buckets, not a fixed segment count. Under typical workloads (lots of buckets, few collisions), most writes are wait-free or near-wait-free.

Treeification applies here too. A heavily-collided bucket becomes a tree, and the synchronized block on the head protects the tree just like it would protect the list.

The equals/hashCode contract

This is the contract that breaks if you ignore it, and HashMap is where it breaks first.

The five rules of equals

1. Reflexive: `x.equals(x)` is always true.
2. Symmetric: `x.equals(y)` returns true iff `y.equals(x)` returns true.
3. Transitive: if `x.equals(y)` and `y.equals(z)`, then `x.equals(z)`.
4. Consistent: repeated calls return the same result, as long as the objects don't change.
5. Null check: `x.equals(null)` returns false (never throws).

The hashCode contract

It must stay consistent during the object's lifetime, as long as fields used in equals don't change. If `x.equals(y)` is true, then `x.hashCode() == y.hashCode()` must be true. If `x.equals(y)` is false, the hash codes don't have to differ, but better hashes (more uniform) make HashMap faster.

The second rule is the killer. Override equals without overriding hashCode and your object disappears from HashMap.

What goes wrong

```
// BROKEN: equals overridden, hashCode not
class User {
    int id;
    String email;

    @Override
    public boolean equals(Object o) {
        if (!(o instanceof User u)) return false;
        return id == u.id && Objects.equals(email, u.email);
    }
    // no hashCode override -> uses Object's identity-based hashCode
}

Map<User, String> map = new HashMap<>();
map.put(new User(1, "a@x.com"), "active");
map.get(new User(1, "a@x.com"));  // returns null
```

Two `new User(1, "a@x.com")` objects are `.equals()` to each other but have different `hashCode()` values. They land in different buckets. The `get()` looks in the wrong bucket and finds nothing.

Correct implementation

```
class User {
    private final int id;
    private final String email;

    public User(int id, String email) {
        this.id = id;
        this.email = email;
    }

    @Override
    public boolean equals(Object o) {
        if (this == o) return true;
        if (!(o instanceof User u)) return false;
        return id == u.id && Objects.equals(email, u.email);
    }

    @Override
    public int hashCode() {
        return Objects.hash(id, email);
    }
}
```

`Objects.hash(...)` boxes the args into an array and combines them. For hot code paths, write it by hand:

```
@Override
public int hashCode() {
    int result = id;
    result = 31 * result + (email == null ? 0 : email.hashCode());
    return result;
}
```

The `31 *` trick is everywhere in the JDK. 31 is an odd prime that the JIT compiles to `(x << 5) - x`.

Shortcuts

- Lombok: `@EqualsAndHashCode` generates both, with config for which fields to include.
- IDE: IntelliJ and Eclipse both generate equals and hashCode from a field selection. Use this if you don't have Lombok.
- Java 16+ records: auto-generate `equals`, `hashCode`, and `toString` correctly.

```
public record User(int id, String email) { }
// equals, hashCode, toString already done. Immutable too.
```

The subclass trap

```
class Point {
    int x, y;
    public boolean equals(Object o) {
        if (!(o instanceof Point p)) return false;
        return x == p.x && y == p.y;
    }
}

class ColoredPoint extends Point {
    String color;
    public boolean equals(Object o) {
        if (!(o instanceof ColoredPoint cp)) return false;
        return x == cp.x && y == cp.y && color.equals(cp.color);
    }
}

Point p = new Point(1, 2);
ColoredPoint cp = new ColoredPoint(1, 2, "red");
p.equals(cp);   // true (cp is a Point with x=1, y=2)
cp.equals(p);   // false (p is not a ColoredPoint)
// symmetry broken
```

Fixes: use `getClass()` instead of `instanceof`, or make equality stop at the class boundary, or favor composition.

Entities and Hibernate

JPA entities are the worst offenders. Auto-generated IDs are null before persist and set afterward. If you use the ID in hashCode, the hash changes after persistence, and any HashMap holding the entity breaks.

Common patterns:

- Use a natural business key (email, SKU) for equals/hashCode. Stable across the lifecycle.
- Or: return a constant from hashCode (e.g., `getClass().hashCode()`) and use ID in equals. Slow lookups but at least the contract holds.
- Or: don't put entities in hash-based collections. Use the ID as the key directly.

Iterator and Iterable

The interfaces

```
public interface Iterable<T> {
    Iterator<T> iterator();
    // also: forEach, spliterator (default methods)
}

public interface Iterator<T> {
    boolean hasNext();
    T next();
    default void remove() { throw new UnsupportedOperationException(); }
    default void forEachRemaining(Consumer<? super T> action) { ... }
}
```

How for-each desugars

```
for (String s : list) {
    System.out.println(s);
}
// compiles to roughly:
Iterator<String> it = list.iterator();
while (it.hasNext()) {
    String s = it.next();
    System.out.println(s);
}
```

For arrays, for-each compiles to an indexed loop instead. No iterator involved.

ListIterator

`ListIterator<T>` is the bidirectional version, only available on `List`:

```
List<String> list = new ArrayList<>(List.of("a", "b", "c"));
ListIterator<String> it = list.listIterator();
while (it.hasNext()) {
    String s = it.next();
    if (s.equals("b")) {
        it.set("B");        // replace current
        it.add("b.5");      // insert after current
    }
}
// list is now [a, B, b.5, c]
```

Methods beyond `Iterator`: `hasPrevious()`, `previous()`, `set(e)`, `add(e)`, `nextIndex()`, `previousIndex()`.

Spliterator

Powers parallel streams. Splits a collection into chunks that can be processed in parallel. You rarely write one yourself, but `list.stream().parallel()` uses it under the hood. Key methods: `tryAdvance`, `trySplit`, `estimateSize`, `characteristics`.

Fail-fast vs fail-safe iterators

The other interview classic.

How fail-fast works

Every `AbstractList` and `AbstractMap` has a `modCount` field. Every structural modification (add, remove, clear) increments it.

When you create an iterator, it captures the current `modCount` as `expectedModCount`. Every `next()` call checks:

```
final void checkForComodification() {
    if (modCount != expectedModCount)
        throw new ConcurrentModificationException();
}
```

If they differ, the iterator throws. This is best-effort. There's no thread-safety guarantee. It's there to catch programming bugs, so don't treat it as a concurrency safeguard.

The broken pattern

```
List<String> list = new ArrayList<>(List.of("a", "b", "c"));
for (String s : list) {
    if (s.equals("b")) {
        list.remove(s);  // ConcurrentModificationException on next iteration
    }
}
```

The fixes

```java
// 1. Use Iterator.remove() (updates expectedModCount)
Iterator<String> it = list.iterator();
while (it.hasNext()) {
    if (it.next().equals("b")) {
        it.remove();
    }
}

// 2. Use removeIf (Java 8+, cleanest)
list.removeIf(s -> s.equals("b"));

// 3. Collect to remove, then removeAll
List<String> toRemove = list.stream()
    .filter(s -> s.equals("b"))
    .toList();
list.removeAll(toRemove);
```

`removeIf` is the right answer 95% of the time.

Fail-safe iterators

`CopyOnWriteArrayList` and `ConcurrentHashMap` iterators never throw `ConcurrentModificationException`. They work on a snapshot:

- `CopyOnWriteArrayList`: iterator holds a reference to the array at the time of iterator creation. Any subsequent mutation creates a new array. The iterator sees the old one.
- `ConcurrentHashMap`: iterator is weakly consistent. It reflects some, but not necessarily all, modifications that happened after creation. Never throws.

The trade-off: you might see stale data. For most read-heavy code that doesn't care about strict consistency, this is fine.

Comparable vs Comparator

The two interfaces

```java
public interface Comparable<T> {
    int compareTo(T o);    // natural ordering, lives on the class
}

public interface Comparator<T> {
    int compare(T a, T b);  // external, you can have many per class
}
```

Return value contract: negative if `a < b`, zero if equal, positive if `a > b`. Magnitude doesn't matter (don't depend on it being -1, 0, 1).

Contract rules

- Antisymmetric: `sgn(x.compareTo(y)) == -sgn(y.compareTo(x))`.
- Transitive: if `x.compareTo(y) > 0` and `y.compareTo(z) > 0`, then `x.compareTo(z) > 0`.
- Consistent: repeated calls return the same result.
- Consistent with equals (strongly recommended): `x.equals(y)` iff `x.compareTo(y) == 0`.

That last one matters because `TreeSet` and `TreeMap` use `compareTo` (not `equals`) to test for duplicates. If your `compareTo` returns 0 for two objects that aren't `.equals()`, only one of them lands in the tree.

```
TreeSet<BigDecimal> set = new TreeSet<>();
set.add(new BigDecimal("1.0"));
set.add(new BigDecimal("1.00"));
set.size();  // 1, because BigDecimal.compareTo treats them as equal,
             // even though .equals() does not
```

Comparator chaining (Java 8+)

This is the API to actually use:

```
List<User> users = ...;

// sort by last name, then first name, descending
users.sort(
    Comparator.comparing(User::getLastName)
        .thenComparing(User::getFirstName)
        .reversed()
);

// nulls last on a nullable field
users.sort(Comparator.comparing(User::getMiddleName,
    Comparator.nullsLast(Comparator.naturalOrder())));

// natural order, reverse order
list.sort(Comparator.naturalOrder());
list.sort(Comparator.reverseOrder());
```

The factory methods read top to bottom in declaration order. `.reversed()` flips the whole chain.

A practical pattern for stable sort by priority then timestamp:

```
Comparator<Task> byPriorityThenTime =
    Comparator.comparingInt(Task::getPriority).reversed()
        .thenComparing(Task::getCreatedAt);
```

`comparingInt`, `comparingLong`, `comparingDouble` avoid boxing.

Hash function quality

The JDK ships well-chosen hash functions for the built-in types. `Integer.hashCode()` returns the int value itself. `String.hashCode()` uses the standard `31 * h + c` rolling hash. `Long.hashCode()` XORs the high and low 32 bits.

These spread well. Your custom types should too.

Bad hash functions

```
// terrible: everything collides
@Override
public int hashCode() { return 0; }

// also bad: low entropy
@Override
public int hashCode() { return id % 16; }
```

If every key hashes to the same bucket, HashMap degrades to a linked list. Java 8 saves you partially: the bucket treeifies, giving O(log n) instead of O(n). But the tree fallback only works if your keys also implement `Comparable`. Otherwise you stay on the linked list.

Default recipe

```
@Override
public int hashCode() {
    return Objects.hash(field1, field2, field3);
}
```

For hot paths where the boxing cost of `Objects.hash` matters, hand-roll:

```
@Override
public int hashCode() {
    int h = 1;
    h = 31 * h + Integer.hashCode(field1);
    h = 31 * h + (field2 == null ? 0 : field2.hashCode());
    h = 31 * h + (field3 == null ? 0 : field3.hashCode());
    return h;
}
```

Same algorithm `Objects.hash` uses, without the array allocation.

Identity-based operations

== vs equals()

```
String a = new String("hello");
String b = new String("hello");
a == b;       // false (different objects)
a.equals(b);  // true (same content)
```

All standard collections use `equals()` and `hashCode()` for membership and lookups. Identity-based collections are the exception.

IdentityHashMap

`IdentityHashMap` uses `==` and `System.identityHashCode()` instead. Two distinct objects with the same content are different keys.

```
IdentityHashMap<String, Integer> m = new IdentityHashMap<>();
m.put(new String("k"), 1);
m.put(new String("k"), 2);
m.size();  // 2, because the two String objects are different references
```

Used for object-graph traversal (serializers, deep-copy frameworks), where two equal-but-distinct objects must be tracked separately. Full coverage in collections-set-map.pdf.

Object header and memory

Every Java object carries a header. On a 64-bit JVM with compressed oops, that's about 12 bytes, padded to the next 8-byte boundary. So even an "empty" object is 16 bytes.

A HashMap `Node<K,V>` holds: `hash` (int), `key` (ref), `value` (ref), `next` (ref). On a 64-bit JVM with compressed oops, that's roughly 32 bytes per entry, plus the entries the keys and values reference.

LinkedList nodes are worse: prev ref, next ref, value ref, plus header. About 24 bytes per node, vs ArrayList which is just a slot in a primitive array (4 or 8 bytes per slot, no header per slot).

When this matters: storing millions of small entries means HashMap memory dominates. If you need primitive keys, use `int[]` or specialized libraries (Eclipse Collections, fastutil, Koloboke). They provide `IntIntMap` without boxing. For cache-heavy code paths, ArrayList traversal is cache-friendly, LinkedList traversal is not.

For typical app code, the overhead doesn't matter. For high-throughput systems, it's the difference between fitting in heap and not.

Best practices

- Treat HashMap keys as immutable. If a field is in `equals` / `hashCode`, don't mutate it after putting the object in a map.
- Always override `hashCode` when you override `equals`. Use `Objects.hash(...)` as the default.
- Use Java 16+ records for value-type classes. They get equals/hashCode/toString right by default.
- Pre-size HashMap when you know the input size: `new HashMap<>(expectedSize * 4 / 3)` to avoid resizes.
- Use `removeIf` for filtering. It avoids `ConcurrentModificationException` and is more readable than iterator loops.
- Make `Comparable` consistent with `equals`, especially for keys in `TreeSet` or `TreeMap`.
- Prefer `Comparator.comparing(...).thenComparing(...)` over hand-written compare methods.
- For business entities with auto-generated IDs, use a natural key in equals/hashCode, or don't put them in hash collections.
- Use `Objects.equals(a, b)` to avoid null checks in equals implementations.

Common gotchas

- Mutating a HashMap key after insertion silently breaks `get()`. Entry is still there, you just can't find it.
- Overriding `equals` without overriding `hashCode` makes objects vanish from HashMap.
- Using `instanceof` (rather than `getClass()`) in equals breaks symmetry if subclasses override equals differently.
- `TreeSet` and `TreeMap` use `compareTo`, not `equals`. A `compareTo` that returns 0 for non-equal objects causes silent dedup.
- `BigDecimal.compareTo` and `BigDecimal.equals` disagree (`1.0` vs `1.00`). Bites people storing BigDecimals in TreeSet.
- `removeIf` on `Collections.synchronizedList` is not actually synchronized. Wrap in `synchronized (list) { ... }`.
- `Iterator.remove()` only works if the underlying collection supports it. `Collections.unmodifiableList(...)` returns iterators where `remove()` throws.
- Fail-fast iterators are best-effort. Don't rely on them to detect concurrent modification across threads. Use concurrent collections.
- `Arrays.asList(arr)` returns a fixed-size list backed by the array. `add` and `remove` throw. Use `new ArrayList<>(Arrays.asList(arr))` if you need mutability.
- Auto-boxed keys (`Map<Integer, String>`) cost more memory than primitive maps. For huge integer-keyed maps, look at Eclipse Collections or fastutil.
- Hash code of 0 is a valid hash. HashMap uses `null` for empty slots, not zero. So `hashCode() == 0` doesn't break anything.
- A class that's `Comparable` to itself but uses fields not in equals gives a TreeSet that contradicts a HashSet for the same elements. Confusing for callers.

Quick reference

Concept	Key Detail
HashMap default capacity	16
HashMap default load factor	0.75
HashMap resize trigger	`size > capacity * loadFactor`
Treeify threshold	8 entries per bucket, table size >= 64
Untreeify threshold	6 entries per bucket
Index formula	`hash & (capacity - 1)` (capacity must be power of 2)
Java 8 hash()	`h ^ (h >>> 16)` (XOR-shift spreads high bits)
equals reflexive	`x.equals(x)` is true
equals symmetric	`x.equals(y)` iff `y.equals(x)`
equals transitive	`x.equals(y)` and `y.equals(z)` imply `x.equals(z)`
hashCode rule	equal objects must have equal hash codes
ConcurrentHashMap (Java 7)	16 segments, segment locks
ConcurrentHashMap (Java 8+)	CAS on empty bucket, synchronized on non-empty bucket
Fail-fast trigger	`modCount != expectedModCount`
Fail-safe iterators	`CopyOnWriteArrayList`, `ConcurrentHashMap`
Best remove during iteration	`list.removeIf(predicate)`
Comparator chaining	`Comparator.comparing(...).thenComparing(...).reversed()`
Records (Java 16+)	auto equals, hashCode, toString
IdentityHashMap	uses `==` and `System.identityHashCode`
Object header	~12-16 bytes per object on 64-bit JVM
HashMap Node size	~32 bytes plus key + value
Primitive map alternative	Eclipse Collections, fastutil, Koloboke